

Abstract Algebra and Number Theory Foundations of Cryptography

Q-1: Prove Fermat's Little Theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simple Fermat test.

Ans:

Fermat's Little Theorem,

If p is prime and $\gcd(a, p) = 1$, then

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof

The non-zero elements, $1, 2, \dots, p-1$

form the multiplicative group $\mathbb{Z}_p^*$

Multiplying each by a only rearranges them:

$$a, 2a, \dots, (p-1)a.$$

$$\text{Thus, } a^{p-1} (p-1)! \equiv (p-1)! \pmod{p}.$$

$$\text{Cancel } (p-1)! \quad a^{p-1} \equiv 1 \pmod{p}$$

Miller Rabin

Write:

$$n-1 = 2^s d$$

calculate $a^d \pmod n$, then repeatedly square if the result becomes $n-1$, n passes; otherwise it is composite.

for 561: $560 = 2^4 (35)$

and,

$$2^{35} \equiv 263, 263^2 \equiv 166, 166^2 \equiv 67, 67^2 \equiv 1 \pmod{561}$$

Since 560 never appears,

561 is composite

But $561 = 3 \times 11 \times 17$

is a Carmichael number, so

$$2^{560} \equiv 1 \pmod{561}.$$

Thus the simple Fermat test is fooled.

Q-2: Prove that $\mathbb{Z}_n$ under addition and multiplication modulo n is a field if and only if n is prime. Illustrate your proof by contrasting multiplicative structure of $\mathbb{Z}_{11}$ (prime modulus) and $\mathbb{Z}_{12}$ (composite modulus).

Ans:

If n is prime, consider $\mathbb{Z}_n = \{0, 1, 2, 3, \dots, n-1\}$ with addition and multiplication modulo n .
For every non-zero element, $a \in \mathbb{Z}_p$

We have, $\gcd(a, p) = 1$

By Bezout's identity,

$$ax + py = 1$$

Taking modulo p , $ax \equiv 1 \pmod{p}$

Therefore every non-zero element has a multiplicative inverse, So, $\boxed{\mathbb{Z}_p \text{ is a field}}$

Suppose, n is composite,

There exist integers a, b with

$$1 < a < n, \quad 1 < b < n$$

such that, $n = ab$

Therefore $ab \equiv 0 \pmod{n}$

But, $a \not\equiv 0 \pmod{n}, b \not\equiv 0 \pmod{n}$.

Thus a, b are zero divisors

A field can not contain non-zero zero divisors

$\mathbb{Z}_n$ is not a field if n is composite

Hence $\mathbb{Z}_n$ is a field $\Leftrightarrow n$ is prime

$\mathbb{Z}(11)$ is a prime,

$$\mathbb{Z}_{11} = \{0, 1, 2, \dots, 10\}$$

It has $11 - 1 = 10$ element

For example, $1 = (9, 0) \text{ b.g. and sh}$

$$2 \times 6 = 12 \equiv 1 \pmod{11}$$

Therefore 6 is a multiplicative inverse of 2

$\mathbb{Z}_{12}$

12 is composite

$$12 = 3 \times 4$$

$$\text{Therefore, } 3 \times 4 = 12 \equiv 0 \pmod{12}$$

$$3 \neq 0, 4 \neq 0$$

So, 3 and 4 are zero divisors

Also 3 has no multiplicative inverse modulo 12

There,

$\mathbb{Z}_{12}$ is not a field

Q-3: Construct $\text{GF}(2^3)$ using the irreducible polynomial x^3+x+1 over $\text{GF}(2)$. Build the multiplication table for a generator element α and determine whether the multiplicative group $\text{GF}(2^3)$ is cyclic.

Ans:

Given that, $f(x) = x^3+x+1$

Over $\text{GF}(2) = \{0, 1\}$

Since the polynomial has degree 3

$\text{GF}(2^3) = \text{GF}(8)$ [$\alpha = x$, Let]

Since, $x^3+x+1=0$, $\alpha^3 = \alpha+1$

Every element can be represented as,

$$a_0 + a_1\alpha + a_2\alpha^2 \quad \text{where } a_0, a_1, a_2 \in \text{GF}(2)$$

$$\text{GF}(8) = \{0, 1, \alpha, \alpha+1, \alpha^2, \alpha^2+1, \alpha^2+\alpha, \alpha^2+\alpha+1\}$$

Power of α using $\alpha^3 = \alpha+1$

$$\alpha^0 = 1$$

$$\alpha^1 = \alpha$$

$$\alpha^2 = \alpha^2$$

$$\alpha^3 = \alpha+1$$

$$\alpha^4 = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha^3 + \alpha^3$$

$$\alpha^6 = \alpha^3 + \alpha^2 + \alpha$$

$$\alpha^7 = 1$$

Multiplication Table:

Using $\alpha^3 = \alpha + 1$

$\times$	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

The multiplicative group

$$|GF(8)^*| = 8 - 1 = 7$$

$$\alpha^7 = 1$$

and no smaller positive power of α equals 1, the order of α is 7.

$$\text{There } GF(8)^* = \langle \alpha \rangle$$

and hence

$$[GF(2^3)^* \text{ is cyclic}]$$

Q-4: Derive the time complexity of modular exponential via square and multiply. Compute $7^{560} \bmod 561$ step by step and interpret the result in light of 561 being a Carmichael number.

Ans:

For modular exponential,

Exponent b has $k = O(\log b)$ bits. Each bit requires one squaring and at most 1 multiplication, totally $O(\log b)$ modular operation

Overall complexity : $O(\log b \cdot \log^2 n)$ or $(O \log^3 n)$
when $b \approx n$.

Compute : $7^{560} \bmod 561$

Exponent $560 = 1000110000_2$ start with $R=1$

1. Bit 1 : $1^2 \times 7 = 7$

2. Bit 0 : $7^2 = 49$

3. Bit 0 : $49^2 = 2401 = 157$

4. Bit 0 : $157^2 = 24649 = 526$

5. Bit 1 : $526^2 \equiv 103 \Rightarrow 103 \times 7 = 721 = 160$

6. Bit 1 : $160^2 \equiv 355 \Rightarrow 355 \times 7 = 2485 \equiv 241$

$$7. \text{Bit } 0: 241^2 = 58081 \equiv 298$$

$$8. \text{Bit } 0: 298^2 = 88804 \equiv 166$$

$$9. \text{Bit } 0: 166^2 = 27556 \equiv 67$$

$$10. \text{Bit } 0: 67^2 = 4489 \equiv 1$$

$$\text{Result: } 7^{560} \equiv 1 \pmod{561}$$

Charmichael Number Interpretation

561 is composite ($3 \times 11 \times 17$)

As a Charmichael number,

it satisfies $a^{n-1} \equiv 1 \pmod{n}$ for

a coprime to n .

Passing Fermat's test ($7^{560} \equiv 1$) gives

a false positive for primality, demonstrating

Why Fermat test is unreliable and robust

alternative like Miller-Rabin are required.

Q-5: Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048 bit and 4096-bit key sizes.

Ans:

AKS vs Miller Rabin

Feature	Miller Rabin	AKS
Type	Probabilistic	Deterministic
Error	Very small	Zero
Speed	Very fast	Slower
RSA Use	Widely used	Rare
2048/4096 bit RSA	Practical	Generally impractical

Miller-Rabin error Probability

For an odd composite number n ,

$$P(\text{false prime}) \leq \left(\frac{1}{4}\right)^k$$

$$\text{or } P(\text{error}) \leq 4^{-k}$$

For example, with 20 independent rounds,
 $4^{-20} = 2^{-40}$

Which is extremely small

Miller Rabin is preferred for RSA:

RSA requires finding very long primes.

For a 2048 bit RSA modulus, the two
 primes p and q are typically around

1024 bits each.

For a 4096 bit RSA, they are around

2048 bits each.

Testing every candidate with a computationally
 expensive deterministic algorithm would be
 inefficient.

$$P(\text{false prime}) \leq \left(\frac{1}{4}\right)^k$$

$$\text{or } P(\text{error}) \leq 4^{-k}$$

Q-6: Derive the extended euclidean algorithm formally then use it to compute $\gcd(240, 46)$ together with the Bezout coefficient x, y such that,

$$240x + 46y = \gcd(240, 46)$$

Ans:

The extended Euclidean Algorithm find integers x, y such that,

$$ax + by = \gcd(a, b)$$

Example:

$$\gcd(240, 46)$$

Euclidean steps:

$$240 = 46(5) + 10$$

$$46 = 10(4) + 6$$

$$10 = 6(1) + 4$$

$$6 = 4(1) + 2$$

$$4 = 2(2) + 0$$

Therefore $\gcd(240, 46) = 2$

Back Substitution

$$2 = 6 - 4$$

$$4 = 10 - 6$$

$$\text{So, } 2 = 6 - (10 - 6) = 2(6) - 1(10)$$

Since,

$$6 = 46 - 4(10)$$

$$2 = 2(46 - 4(10)) - 1(10)$$

$$= 2(46) - 9(10)$$

Since,

$$10 = 240 - 5(46)$$

$$2 = 2(46) - 9(240 - 5(46))$$

$$= 47(46) - 9(240)$$

Therefore,

$$240(-9) + 46(47) = 2$$

Comparing with,

$$240x + 46y = \gcd(240, 46)$$

Therefore, $x = -9$,

$$y = 47$$

Q-7: State and Prove the Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve,

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 4 \pmod{7}, x \equiv 5 \pmod{11}$$

Ans:

Statement:

If $m_1, m_2, \dots, m_k$

are pairwise coprime then the system

$$x \equiv a_i \pmod{m_i}$$

has a unique solution modulo

$$M = m_1 m_2 \dots m_k$$

Proof idea

Because the moduli are pairwise coprime each m_i has an inverse modulo the other product.

Therefore a solution exist.

Any two solutions differ by a multiple of

$$M = m_1 m_2 \dots m_k,$$

So the solution is unique modulo M .

Solve: State and prove the Chinese Remainder Theorem (existence and uniqueness) and apply it to solve.

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 4 \pmod{7}$$

$$x \equiv 5 \pmod{11}$$

Ans:

Total modulus:

$$M = 3 \times 5 \times 7 \times 11 = 1155$$

i	m_i	$M_i = M/m_i$	$M_i \pmod{m_i}$	inverse y_i	$a_i M_i y_i$
1	3	385	1	1	770
2	5	231	1	1	693
3	7	165	4	2	1320
4	11	105	6	2	1050

$$\text{Sum} = 770 + 693 + 1320 + 1050 = 3833$$

$$\text{Reduce mod 1155: } 3833 - 3 \cdot 1155 = 368$$

$$x \equiv 368 \pmod{1155}$$

Check:

$$368 = 3 \cdot 122 + 2, = 5 \cdot 73 + 3, = 7 \cdot 52 + 4, = 11 \cdot 33 + 5$$

8. Derive the CRT-based RSA decryption/signing optimization from first principle (computing d_p , d_q and q_{inv}) and demonstrate the $\sim 4\times$ speed-up with a fully worked numerical example using small primes p, q .

Ans:

In RSA,

$$n = pq$$

and decryption is $m = c^d \pmod{n}$

CRT perform two smaller computation

$$m_p = c^{d_p} \pmod{p}$$

$$m_q = c^{d_q} \pmod{q}$$

Where,

$$d_p = d \pmod{p-1}$$

$$d_q = d \pmod{q-1}$$

$$\text{and } d_{inv} = q^{-1} \pmod{p}$$

Then combine:

$$h = q_{inv} (m_p - m_q) \pmod{p}$$

$$m = m_q + qh$$

Example:

Take: $p=5$, $q=11$, $n=55$

Let, $d=27$, $c=7$

Then, $d_p = 27 \bmod 4 = 3$

$d_q = 27 \bmod 10 = 7$

Calculate:

$$m_p = 7^3 \bmod 5 = 3$$

$$m_q = 7^7 \bmod 11 = 6$$

Find, $a_{inv} = 11^{-1} \bmod 5 = 1$

Thus, $h = 1(3-6) \bmod 5 = 2$

Finally $m = (6 + 11(2)) = 28$

So, $m = 28$

Without CRT: one exponential modulo n

With CRT: two exponentiations modulo approximately $n^{1/2}$

Smaller numbers make multiplication much faster giving approximately

4 x speedup

Q-9: Explain the Bellcore fault-injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n_1 and what countermeasure prevent it.

Ans:

Mechanism:

Suppose an attacker induces a hardware fault during a computation of S_p , producing a faulty signature S_p^1 , while S_q is calculated correctly.

1. Faulty Signature Construction:

The output is S^1 , where $S^1 \equiv S_p^1 \pmod{p}$ (incorrect)

and, $S^1 \equiv S_q \pmod{q}$ (correct).

2. Key Factorization:

Consider the difference. $(S^1 - M)$:

• Modulo q : $S^1 - M \equiv S_q^e - M \equiv 0 \pmod{q} \Rightarrow q | (S^1 - M)$

• Modulo p : $S^1 - M \not\equiv 0 \pmod{p} \Rightarrow p \nmid (S^1 - M)$.

3. Extracting q :

$$\gcd(S^1 - M, N) = q$$

A single faulty signature completely factors N using simple gcd computation.

Counter measures:

- Verification before Output: compute $S^e \pmod{N}$ and verify it equals M before sending the signature. If incorrect suppress output.

- Randomized Blinding: Introduce random salt-blinding factor to signature steps so intermediate faulty result do not map to linear relation.

- Double Computation: Compute S_p or S_q twice and compare results before CRT combination.

Q-10: Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's Little Theorem based exponentiation and discuss trade-off between the two approaches for large moduli.

Ans:

Find the inverse of $3 \pmod{7}$

We need $3x \equiv 1 \pmod{7}$.

Method 1: Extended Euclidean Algorithm

$$7 = 3(2) + 1$$

$$\text{Therefore, } 1 = 7 - 2(3)$$

$$\text{So, } 1 = (-2) \times 3 + 1(7)$$

hence,

$$3^{-1} \equiv -2 \equiv 5 \pmod{7}$$

Check

$$3 \times 5 = 15 \equiv 1 \pmod{7}$$

Method 2: Fermat's Little Theorem

Since 7 is prime,

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore, $a^2 \equiv a^{p-2} \pmod{p}$

For, $a=3$ $p=7$

$$3^{-1} \equiv 3^5 \pmod{7}$$

$$3^5 = 243$$

and $243 \bmod 7 = 5$

Thus $3^{-1} \equiv 5 \pmod{7}$

Comparison

Method	Advantage
Extended Euclidean	Direct and efficient
Fermat	Easy when modulus is prime
Large modulus	Euclidean is generally preferred for finding an inverse.

11. Design and fully execute a playfair cipher encryption and decryption of a phrase of our choosing (minimum 16 letters) with a keyword of choosing. Present the complete 5x5 key matrix as a table and show every digraph substitution step.

Ans:

Keyword: MONARCHY

Plaintext: Instruments (INSTRUMENTS)

16 + Letters use:

INSTRUMENTS ARE READY

Create 5x5 matrix

Keyword = MONARCHY

Remove repeated letters and fill remaining alphabet:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

Make digraphs

IN, ST RU ME NT SA RE EA DY

Playfair rules:

1. Same row: move one position right
2. Same column: move one position down
3. Different row/column: form a rectangle and take other corners.

Example Substitution

IN, ST RU ME NT SA RE EA DY

I = row 3 column 4

N = row 1 column 3

Rectangle gives:

I → G, N → A

ST ⇒ S(4,4) T(4,5) same row:

Therefore ST → TL

The remaining pairs are proceed using the same rules.

12. Critically analyze the structural weakness of the playfair cipher against digraph frequency analysis and explain why it remains vulnerable to known-plaintext attacks despite being far stronger than monoalphabetic substitution.

Ans:

Critical Analysis:

The playfair cipher was a significant advancement over monoalphabetic substitution because it encrypts digraphs, effectively flattening single letter frequency distribution. However, it remains vulnerable to cryptanalysis for

several reasons:

1. Digraph Frequency Analysis: While single letter frequency is hidden, the ciphertext preserves the frequency of digraphs. In any language, certain pairs of letters occur much more frequently than others.

2. Common letters pairs can still be recognized statistically.
3. It preserve some digraph frequency pattern.
4. Repeated plaintext pattern may produce repeated ciphertext patterns.
5. The key space is limited compared with modern block cipher.
6. It can be attacked using known plaintext

A known pair of plaintext attack:

- A known pair of plaintext digraphs directly reveals geometry relative position in the 5×5 matrix.

If an attacker knows some plaintext and its ciphertext, they can learn relationship in the playfair cipher matrix and gradually recover the key.

Q-13 : Derive the and trace two full round of a simplified Feistel-structure cipher (DES-like), showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAAAAAA$, $R_0 = 0xF0F0F0F0$, $K_1 = 0x0F0F0F0F$, $K_2 = 0x33333333$ using XOR as the round function.

Ans:

A Feistel round is defined as:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Hence,

$$F(R, K) = R \oplus K$$

Given,

$$L_0 = 0xAAAAAAAA$$

$$R_0 = 0xF0F0F0F0$$

$$K_1 = 0x0F0F0F0F$$

$$K_2 = 0x33333333$$

Round : 1

$$L_1 = R_0$$

$$L_1 = F0F0F0F0$$

Calculate

$$\begin{aligned} F(R_0, K_1) &= F0F0F0F0 \oplus 0F0F0F0F \\ &= FFFFFFFF \end{aligned}$$

Therefore,

$$R_1 = L_0 \oplus F(R_0, K_1)$$

$$= \text{AAAAAAAA} \oplus \text{FFFFFFFF}$$

$$= 55555555$$

$$\text{So, } L_1 = \text{F0F0F0F0}, R_1 = 55555555$$

Round 2:

$$L_2 = R_1 = 55555555$$

$$\begin{aligned} \text{Calculate } F(R_1, K_2) &= 55555555 \oplus 33333333 \\ &= 66666666 \end{aligned}$$

Therefore,

$$R_2 = L_1 \oplus F(R_1, K_2)$$

$$= \text{F0F0F0F0} \oplus 66666666$$

$$= 96969696$$

Final Answer: $R_2 = 96969696$

$$L_2 = 55555555$$

14. Formally define the avalanche effect in block ciphers. Using a toy 8 bit substitution-permutation network of your own construction demonstrate how flipping a single input bit changes at least half of the output bits and explain why this property is essential for cryptographic security.

Ans:

Definition:

The avalanche effect means that changing one input bit should change many output bits. Ideally, about half of the output bits should change.

For an 8-bit Output

At least about 4 bits should change when one input bit changes.

Example:

Suppose an 8 bit substitution gives:

Input:

10110010

↓

S-Box

↓

Output : 01101101

Now flip one input bit,

10110010

↓

10110011

Suppose the new output is:

11001011

Compare:

Old : 01101101

New : 11001011

↑ ↑ ↑

Avalanche effect makes it difficult for attackers to determine the relationship between plaintext and ciphertext.

Q-15: Evaluate the brute-force resistance of an 8 character alphanumeric password (62 character alphabet) versus a 6 digit pin, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst case cracking time for both and discuss the implications for password policy design.

Ans:

Given cracking rate 10^9 guesses/second

6 digit PIN

There are: $10^6 = 1,000,000$

possible PINs.

Worst case:

$$\frac{10^6}{10^9} = 0.001 \text{ sec}$$

$$= 1 \text{ millisecond}$$

Expected case

On average, half the possibilities:

$$\frac{10^6}{2 \times 10^9} = 0.0005 \text{ sec}$$

$$= 0.5 \text{ ms}$$

8 character alphanumeric password

There are 62 possible characters

$$62^8$$

$$62^8 = 218,340,105,584,896$$

possibilities

Worst case:

$$218,340,105,584,896$$

$$\approx 218,340 \text{ seconds}$$

$$\text{Approximately } 60.65 \text{ hours} \approx 2.53 \text{ days}$$

$$\text{Expected case: } \frac{2.53}{2} \approx 1.26 \text{ days}$$

Comparison

Credential	Possibilities	Worst Case
6 Digit PIN	10^6	1 ms
8-char alphanumeric	62^8	$\approx 2.53 \text{ days}$

Q: 16: Compare the error propagation and parallelizability characteristics of ECB, CBC, CFB, OFB and CTR modes using a labelled diagram for each. Recommend the most suitable mode for a) encrypting a live video stream over a lossy network and b) encrypting a static disk image with justification

Ans:

Comparison:

Mode	Parallel encryption	Parallel decryption	Error Propagation	Requires Unique IV
ECB	Yes	Yes	Single block	No
CBC	No	Yes	Corrupt current + next block	Yes
CFB	No	Yes	Corrupt current + next block	Yes
OFB	No	No	Single bit	Yes
CTR	Yes	Yes	Single bit	Yes

Diagram:

ECB:

$P_1 \rightarrow E \rightarrow C_1$

$P_2 \rightarrow E \rightarrow C_2$

$P_3 \rightarrow E \rightarrow C_3$

CBC:

$$P1 \rightarrow \text{XOR} \rightarrow E \rightarrow C1 \rightarrow \text{XOR} \rightarrow E \rightarrow C2$$

CFB

$$IV \rightarrow E \rightarrow \text{XOR } P1 \rightarrow C1$$

$$E \rightarrow \text{XOR } P2 \rightarrow C2$$

OFB:

$$IV \rightarrow E \rightarrow E \rightarrow E$$

$$\begin{array}{cc} \downarrow & \downarrow \\ \text{XOR} & \text{XOR} \end{array}$$

$$\begin{array}{cc} \downarrow & \downarrow \\ C1 & C2 \end{array}$$

CTR

$$\text{Counter} \rightarrow E \rightarrow \text{XOR } P1 \rightarrow C1$$

$$\text{Counter} + 1 \rightarrow E \rightarrow \text{XOR } P2 \rightarrow C2$$

Recommendation:

- Live video : OFB $\Rightarrow$ bit errors, do not spread to other bits.
- Static disk image $\rightarrow$ CTR $\rightarrow$ encryption/decryption can be parallel and errors affect only the corresponding bits.

Q:17: Using the Partial S AES-S-Box below, perform the SubBytes transformation on the state word $[0x23, 0x4C, 0x19]$, then supply shift rows to a 4×4 matrix that has its first row (fill remaining row with 0x00 for the purpose of the exercise. Report the resulting matrix.

Row \ col $\rightarrow$	3	4	7	9	A	C
1	6D			C6		
2	D4					
4		A1				2E
A			63		D2	

Given State word:

$[23, A7, 4C, 19]$

From the S-Box:

$23 \rightarrow DA$

$A7 \rightarrow 63$

$4C \rightarrow 2E$

$19 \rightarrow C6$

Therefore after Subbytes:

$[D4, 63, 2E, C6]$

Complete State:

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

Shift Rows

Row 1 $\rightarrow$ no shift

Row 2 $\rightarrow$ shift left 1

Row - 3 $\rightarrow$ shift left 2

Row $\rightarrow$ 4 $\rightarrow$ shift left 3

Since the last three rows contain only zero,
the matrix remains:

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

Q:18: Derive the AES $\rightarrow$ 128 Rijndael key schedule and compute the first two round keys (K_1, K_2) from a given 128-bit cipher key of your choosing, showing the RotWord, SubWord and Rcon steps explicitly.

Ans:

Chose the standard 128 bit key:

2B7E151628AED2A6ABF7158809CF4F3C

Divide into Four words:

$$W_0 = 2B7E1516$$

$$W_1 = 28AED2A6$$

$$W_2 = ABF71588$$

$$W_3 = 09CF4F3C$$

Round 1

Rot Word

$$09CF4F3C \rightarrow CF4F3C09$$

Subword

$$CF4F3C09 \rightarrow 8A84EB01$$

XOR Rcon

$$8A84EB01 \oplus 01000000 = 8B84EB01$$

$$\begin{aligned} \text{Then } W_4 &= A0 \cdot W_0 \oplus 8B84EB01 \\ &= A0FAFE17 \end{aligned}$$

Next, $w_5 = w_1 \oplus w_4 = 88542CB1$

$$w_6 = w_2 \oplus w_5 = 23A33939$$

$$w_7 = w_3 \oplus w_6 = 2A6C7605$$

Therefore,

$$K_1 = A0FAFE1788542CB123A339392A6C7605$$

Round 2:

Rotword : $2A6C7605 \rightarrow 6C76052A$

Subword : $6C76052A \rightarrow 50386BE5$

$$\begin{aligned} \text{Rcon: } 50386BE5 \oplus 0200000000 \\ = 52386BE5 \end{aligned}$$

Then,

$$w_8 = w_4 \oplus 52386BE5 = F2C295F2$$

Counting gives,

$$w_9 = 7A96B943$$

$$w_{10} = 5935807A$$

$$w_{11} = 7359F67F$$

Therefore,

$$K_2 = F2C295F27A96B9435935807A7359F67F$$

Q-19: Explain the MixColumn transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed MixColumns matrix, showing the $GF(2^8)$ arithmetic for each byte.

Ans:

MixColumns uses:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Over $GF(2^8)$

Take the example column:

$$\begin{bmatrix} DB \\ 13 \\ 53 \\ 45 \end{bmatrix}$$

The first output byte is:

$$(02 \times DB) \oplus (03 \times 13) \oplus 53 \oplus 45$$

Using GF arithmetic:

$$02 \times DB = AD$$

$$03 \times 13 = 35$$

Therefore,

$$AD \oplus 35 \oplus 53 \oplus 45 = [8E]$$

The complete result is:

$$\begin{bmatrix} 8E \\ AD \\ A1 \\ BC \end{bmatrix}$$

Qn-20: Critically evaluate AES' resistance to side channel attacks. Specifically timing analysis (e.g. cache timing attacks on table-based S-box implementations) and differential power analysis. Describe at least two concrete countermeasures (e.g. constant time S-box masking and their performance trade-offs).

Ans:

Side channel attacks the AES mathematics directly. They use information such as time or power consumption.

1. Timing Attack:

A table based S box may access different memory location depending on secret data.

Secret key



AES-S-box lookup



Different memory access



Different timing



Key information

2. Different Power Analysis (DPA)

The attacker measure power consumption many times and uses statistical analysis to discover key information.

Counter measures:

1. Constant time S-box

Use implementation whose complex execution time does not depend on secret data.

2. Masking

Randomly mask intermediate values so power consumption does not directly reveal the secret.

Trade-off

Countermeasure	Advantage	Disadvantage
Constant-time	Prevents timing leakage	May reduce speed
Masking	Strong against power analysis	More computation and complexity

Q-21: Derive the RSA key-generation, encryption and decryption procedure from Euler's Theorem, and Prove correctness of decryption (i.e. such that $(Me)^d \equiv M \pmod{n}$) for the general case, including the case $\gcd(M, n) \neq 1$

Ans:

RSA uses Euler's Theorem

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

$$\text{When } \gcd(a, n) = 1$$

Key generation,

Choose two primes, p, q

Calculate $n = pq$

$$\phi(n) = (p-1)(q-1)$$

Choose e such that, $\gcd(e, \phi(n)) = 1$

Find d : $ed \equiv 1 \pmod{\phi(n)}$

Public Key : (e, n)

Private Key : (d, n)

Encryption : $C = M^e \pmod{n}$

Decryption : $M = C^d \pmod{n}$

Since, $ed = 1 + k\phi(n)$

For $\gcd(M, n) = 1$,

$$M^{ed} = M^{1+k\phi(n)} = M(M^{\phi(n)})^k \equiv M \pmod{n}$$

For the case $\gcd(m, n) \neq 1$ use the prime

factor p, q . RSA correctness follows:

modulo both p and q . by the Chinese

Remainder Theorem,

$$C^d \equiv M \pmod{n}$$

Q-22: Given primes $p=11, q=11$ generate

a full RSA keypair (choosing a valid public

exponent e) then sign a message hash

$H(M)=23$ and verify the signature,

showing all intermediate modular

exponential steps:

Public Key: (e, n)

Private Key: (d, n)

Encryption: $C = M^e \pmod{n}$

Decryption: $M = C^d \pmod{n}$

Ans:

Given,

step 1:

$$P=11, q=13$$

$$\text{Calculate, } n = 11 \times 13 = 143$$

Step-2: Euler phi

$$\phi(n) = 10 \times 12 = 120$$

Choose, $e = 7$

$$\text{because } \gcd(7, 120) = 1$$

$$\text{Find } d: 7d \equiv 1 \pmod{120}$$

$$d = 103$$

because

$$7 \times 103 = 721 \equiv 1 \pmod{120}$$

$$\text{Thus, Public Key} = (7, 143)$$

$$\text{Private Key} = (103, 143)$$

Signing

$$\text{Given hash: } H(M) = 23$$

$$\text{Signature: } S = 23^{103} \pmod{143}$$

Using repeated squaring:

$$\begin{array}{l|l|l} 23^2 \equiv 100 & 23^8 \equiv 100 & 23^{32} \equiv 100 \\ 23^4 \equiv 133 & 23^{16} \equiv 133 & 23^{64} \equiv 133 \end{array}$$

and, $103 = 64 + 32 + 4 + 2 + 1$

Therefore, $S = 23^{103} \equiv 23 \pmod{143}$

So, $S = 23$

Verification:

$S^e = 23^7 \pmod{143}$

Since, $23^7 \equiv 23 \pmod{143}$

We get $23 = H(M)$

Therefore the signature is valid.

Q-23: Derive a general formula for IPsec ESP overhead percentage in tunnel mode as a function of payload size, header size, authentication data size and padding then evaluate it for a 1200 byte payload with an 8 byte ESP header, 16 byte of authentication (data and 2 bytes of padding). Discuss how this overhead scales for small IoT-sized packet versus large payloads.

Ans:Let, payload = P bytesESP header = H bytesAuthentication data = A bytesPadding = Pad bytesTotal Overhead, $O = H + A + Pad$ Overhead Percentage, $Overhead\% = \frac{H + A + Pad}{P} \times 100$ Given, $P = 1200$ $H = 8, A = 16, Pad = 2$ Therefore, $O = 8 + 16 + 2 = 26$ bytes $Overhead = \frac{26}{1200} \times 100 \approx 2.17\%$ IoT vs Large Payloads:

- Large Payloads: Fixed protocol overhead ($\approx 26B$) represent a tiny fraction of total bandwidth resulting in minimal throughput impact.
- Small IoT Payloads: Fixed 26B overhead dominate the transmission. These causes severe bandwidth degradation and high energy consumption in battery powered IoT devices.

Q-24: Execute a full Diffie-Hellman key exchange with $p=23$, $g=5$, private keys $a=6$ (Ateya), and badol $b=15$, confirming both parties derive the same shared secret. Then explain the man-in-the-middle vulnerability of unauthenticated DH and describe how an authenticated variant (eq. the station-to-station protocol) mitigates it.

Ans:

Given, $P=23$, $g=5$

Ateya's private key: $a=6$

Badol's private key: $b=15$

Step 1: Ateya's public key

$$A = g^a \mod P$$

$$A = 5^6 \mod 23$$

$$A = 8$$

Step 2: Badol's public key

$$B = g^b \mod P$$

$$B = 5^{15} \mod 23$$

$$B = 19$$

Step 3:

Ateya's calculations:

$$K = B^a \bmod p$$

$$K = 19^6 \bmod 23$$

$$K = 2$$

Bob's calculates:

$$K = A^b \bmod p$$

$$K = 8^{15} \bmod 23$$

$$K = 2$$

Therefore

$$\text{Shared secret} = 2$$

Man in the middle attack:

Unauthenticated DH does not prove who sent the public key. An attacker can replace public keys and establish separate secrets with both parties.

Solution: Authenticated DH, such as certificate/signatures in protocols such as authenticated key exchange, prevents the attacker from substituting public keys unnoticed.

Q-25: Compare the computational cost of establishing a shared key via RSA key transport versus Diffie Hellman key agreement at an equivalent security level (e.g. 3072-bit RSA/DH versus 256 bit ECDH). Discuss why forward secrecy favors (EC)DHE over RSA key transport in modern TLS.

Ans:

RSA key transport -

RSA uses a large key, such as:

3072-bit RSA

The client encrypts a secret using the server's RSA public key.

EDCh

EDCh can provide comparable security using a much smaller key such as:

256-bit EDCH

Comparison		
Feature	RSA	EDCH
Example size	3072 bit	256 bit
computation	Large integer operations	smaller elliptic-curve operation
Key exchange	Key transport	key agreement
Forward secrecy	Not for RSA key transport	Yes with ephemeral EDCH

Forward Secrecy

With ephemeral EDCH a fresh temporary key is generated for each session.

RSA key transport does not provide this property: if long term RSA private key is later compromised and recorded encrypted sessions are available - past transported secret may be recoverable.

Therefore, modern TLS uses ephemeral (EC)DHE for forward secrecy.

Q-26: In an ElGamal scheme with $p=467$, $g=2$, private key $x=127$ message hash $H(M)=100$, and nonce, $k=213$ (with $\gcd(k, p-1)=1$):

compute the public key, generate the signature (r, s) and verify it.

Ans:

Given:

$$P=467, g=2, x=127, H(M)=100, k=213$$

1. Public Key, $y = g^x \mod p$

$$y = 2^{127} \mod 467$$

$$y = 132$$

Public Key: $(467, 2, 132)$

2. Signature

First $r = g^k \mod p$

$$r = 2^{213} \mod 467$$

$$r = 29$$

Find $k^{-1} \pmod{466}$:

$$213^{-1} \equiv 431 \pmod{466}$$

Then,

$$S = k^{-1} (H(M) - x\pi) \pmod{466}$$

$$S = 431 (100 - 127 \times 29) \pmod{466}$$

$$S = 51$$

Therefore,

$$(\pi, S) = (29, 51)$$

Verification:

Check

$$g^{H(M)} (y^{\pi} x^S) \pmod{p} \equiv y^{\pi} x^S \pmod{p}$$

Left side:

$$2^{100} \equiv 189 \pmod{467}$$

Right side:

$$132^{29} 29^{51} \equiv 189 \pmod{467}$$

Therefore,

Signature is valid

Q-27: Prove algebraically that the ElGamal signature verification equation $g^{H(M)} \equiv y^r r^s \pmod{p}$ holds for a correctly generated signature, starting from the definition $S \equiv k^{-1}(H(M) - xr) \pmod{p-1}$.

Ans:

Signature definition:

$$S \equiv k^{-1}(H(M) - xr) \pmod{p-1}$$

Multiply by k :

$$kS \equiv H(M) - xr \pmod{p-1}$$

Therefore,

$$H(M) \equiv xr + kS \pmod{p-1}$$

Raise g to both sides:

$$g^{H(M)} \equiv g^{xr + kS} \pmod{p}$$

Since, $y = g^x$, $r = g^k$

$$\text{we got, } g^{H(M)} \equiv (g^x)^r (g^k)^S$$

$$\boxed{g^{H(M)} \equiv y^r r^S \pmod{p}}$$

Thus the verification equation is proved.

28. On the elliptic curve $y^2 \equiv x^3 + x + 1 \pmod{23}$:
 verify that $P = (3, 10)$ lies on the curve, compute
 $2P$ via point doubling and compute $P+Q$
 for $Q = (9, 7)$, showing the full modular arithmetic
 derivation of the slope in each case.

Ans:

Given,

$$y^2 \equiv x^3 + x + 1 \pmod{23}$$

$$\text{and } P = (3, 10), \quad Q = (9, 7)$$

Check P

$$\text{Left} = 10^2 = 100 \equiv 8 \pmod{23}$$

$$\text{Right} = 3^3 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

Therefore P lies on the curve

Calculate $2P$

$$\text{Slope : } \lambda = \frac{3x_1^2 + a}{2y_1}$$

Here, $a = 1$,

$$\lambda = \frac{3(3^2) + 1}{20} = \frac{28}{20}$$

$$28 \equiv 5, \quad 20^{-1} \equiv 15 \pmod{23}$$

$$\text{So, } \boxed{\lambda = 6}$$

Now, $x_3 = \lambda^2 - 2x_1$
 $= 36 - 6 = 30 \equiv 7$
 $y_3 = \lambda(x_1 - x_3) - y_1$
 $= 6(3 - 7) - 10 = -34 \equiv 12$

Therefore $\boxed{2P = 7, 12}$

Calculate $P+Q$

Slope: $\lambda = \frac{y_2 - y_1}{x_2 - x_1} = \frac{10 - 3}{7 - 3} = \frac{7}{4}$

Since $6^{-1} \equiv 4 \pmod{23}$

$\lambda = -3(4) \equiv 11$

Now, $x_3 = 11^2 - 3 - 9 = 109 \equiv 17$

$y_3 = 11(3 - 17) - 10 = -164 \equiv 20$

Therefore $\boxed{P+Q = 17, 20}$

Q-20: On the curve $y^2 = x^3 + 7x + 10 \pmod{37}$ with base point $G = (2, 5)$ of order $n=19$ and private key $d=9$; compute the public key, $Q = dG$, generate an ECDSA signature (r, s) for $H(M)=8$ using nonce $(k=3)$ and verify it against Q .

Ans;

Given :

$$y^2 = x^3 + 7x + 10 \pmod{37}$$

$$G = (2, 5), n = 19, d = 9$$

$$H(M) = 8, k = 3$$

The question states that G has order 19 but direct calculation gives:

$$19G \neq 0$$

In fact $G = (2, 5)$ has order 44, not 19. Therefore, the given data are inconsistent with standard ECDSA.

$$n = 19$$

Public Key

$$Q = dG = 9G$$

$$Q = (23, 11)$$

Signature: $kG = 3G (28, 5)$

Therefore, $r = 28 \bmod 19$

$$r = 9$$

Since, $3^{-1} \equiv 13 \pmod{19}$

$$s = k^{-1} \cdot (H(M) + dr) \pmod{19}$$

$$= 13 (8 + 0 \times 9) \pmod{19}$$

$$\boxed{s = 17}$$

Thus, $\boxed{r, s = 9, 17}$

However standard ECDSA verification with the supplied curve/base point does not return $r=9$ because the stated order-19 condition is false.

Q-30: Compare the key size equality between RSA, ElGamal and ECC (eq. 256 bit ECC $\approx$ 3072-bit RSA / ElGamal at the ~ 128 bit security level) and explain why ECC is preferred for resource constrained devices such as smart card and IoT sensors.

Ans:

At approximately 128 bit security:

System	Approximate key size
RSA	3072 bits
ElGamal	3072 bits
ECC	256 bits

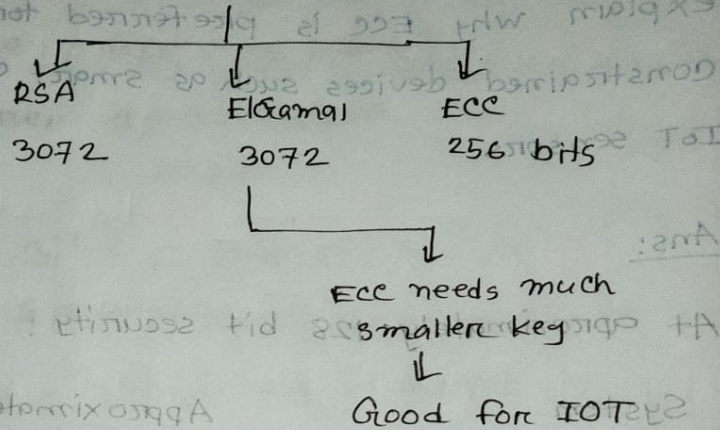
Why ECC is preferred

ECC provide similar security with much smaller keys.

Therefore it needs:

- Less memory
- Less storage
- Less bandwidth

- Less computation
- Lower power consumption
- 128 bit security



Qn-31: Formally derive preimage resistance, second preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday attack complexity ($\approx 2^{n/2}$) for finding a collision in an n bit-bit hash, and explain why this motivates 256 bit (rather than 128 bit) output length for modern hash function.

Ans: Hash Function Security

A cryptographic hash converts a message into a fixed size value.

$$H(M) = \text{hash}$$

Three Resistance:

1. Preimage resistance

Given, h is difficult to find M such that,

$$H(M) = h$$

2. Second Preimage resistance:

Given M_1 is difficult to find, $M_2 \neq M_1$ such that,

$$H(M_1) = H(M_2)$$

3. Collision resistance:

It is difficult to find any two different message M_1, M_2 such that,

$$H(M_1) = H(M_2)$$

Birthday attack

For an n bit hash there are 2^n possible output.

By the birthday paradox, a collision needs approximately $2^{n/2}$ hash collection.

For 128-bit: $2^{128/2} = 2^{64}$

For 256 bit: $2^{256/2} = 2^{128}$

Thus 256 bit hashes give about 128 bit collision security.

$$H(M)H = (SM)H$$

Q-32: Trace the Merkle-Damgard construction

with a diagram showing message blocks, comparison function calls and changing variables.

Explain the length extension attack that this construction is vulnerable to and how HMAC and SHA-3 (sponge construction) avoid it.

Ans:

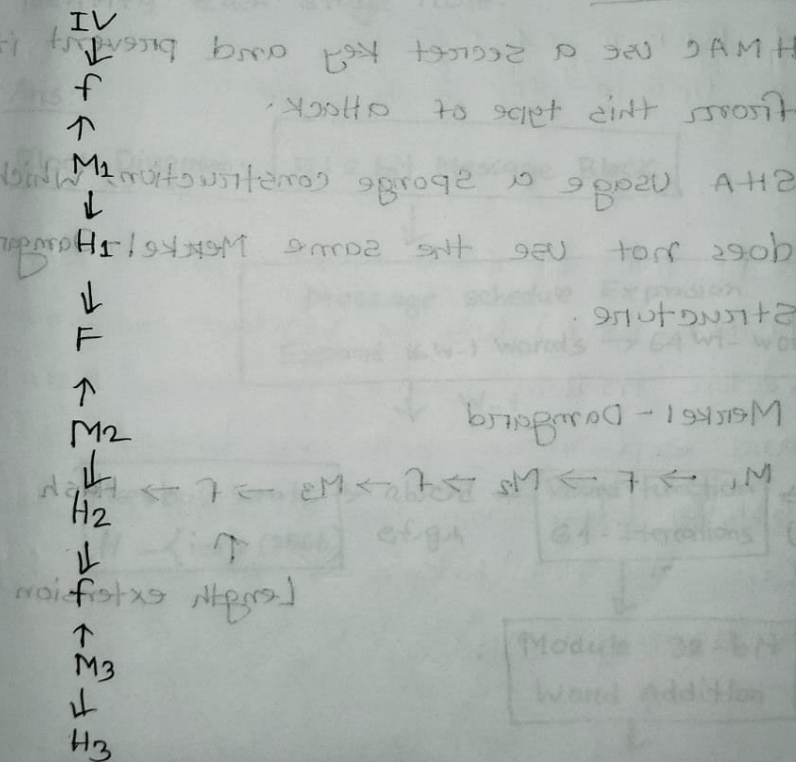
Merkle Damgard Construction

The message is divide into blocks:

$M_1, M_2, \dots, M_t$

and proceed through a compression function

Diagram:



Final Value: $H(M) = H(t)$

Length extension attack

If an attacker knows:

$$H(M)$$

they can sometime calculate the hash

$$M \parallel \text{padding} \parallel M'$$

without knowing the original message M .

Protection:

HMAC use a secret key and prevent it from this type of attack.

SHA usage a sponge construction, which does not use the same Merkle-Damgard structure.

Merkel-Damgard

$$M_1 \rightarrow f \rightarrow M_2 \rightarrow f \rightarrow M_3 \rightarrow f \rightarrow \text{Hash}$$

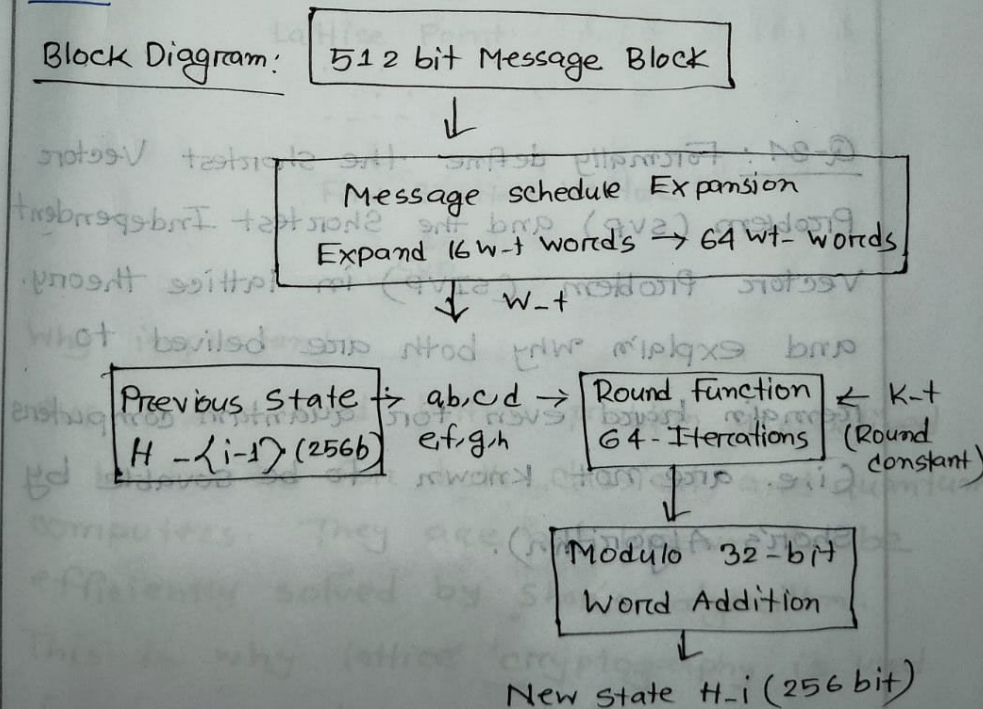
↑
Length extension

$$H(M \parallel \text{padding} \parallel M') = H(M)$$

Q-33: Explain the high level internal structure of SHA-256 compression function, —message schedule expansion, the eight working variables, and the round function — using a labelled block diagram. You are not required to reproduce the exact logic but must correctly identify each stage role.

Ans:

Block Diagram:



Main Steps:

1. Message is divided into 512 bit block
2. Create 64 words $W_0, \dots, W_{63}$ through message scheduling
3. Initialize 8 working variables
a, b, c, d, e, f, g, h
4. Perform 64 round
5. Update the hash state

Block Diagram: 512 bit Message Block

Q-34: Formally define the shortest Vector

Problem (SVP) and the Shortest Independent

Vector Problem (SIVP) in lattice theory.

and explain why both are believed to

remain hard even for quantum computers

(i.e. are not known to be solvable by

Shor's Algorithm).

Ans: Let, $B = \{b_i | i \in \mathbb{Z}^m\}$ be a lattice

Shortest Vector Problem (SVP)

Given a lattice, find the shortest non-zero lattice vectors.

Find $v \neq 0$ with minimum $\|v\|$

Shortest Independent Vector Problem (SIVP)

Find a set of n linearly independent lattice vectors whose length are as small as possible

Lattice Point

$$\dots + a_i + \dots = t$$

↓

Find Shortest vector

↓

SVP

Why important:

SVP and related lattice problems are believed to be hard for both classical and quantum computers. They are not known to be efficiently solved by Shor's algorithm.

This is why lattice cryptography is used in post-quantum cryptography.

Q-35: Explain the core idea behind CRYSTALS-Kyber's Module-Learning-with-Errors (Module-LWE) key encapsulation mechanism including how noise is used to hide the secret and why this construction resists both classical and quantum attacks.

Ans:

The basic idea in Module LWE.

A public equation has form,

$$t = As + e$$

where, A = public matrix

s = secret

e = small random noise

The noise e hides the secrets.

Simple idea

Secret s

+
Small noise e

↓
 $As + e$

↓
Public information

An attacker sees the noisy equation but has difficulty recovering.

The legitimate receiver has secret information that allows correct decapsulation and recovery of the shared key.

Why Secure:

The underlying Module-LWE problem is believed to be difficult for both:

1. Classical computers
2. Quantum Computers

Therefore, ML-KEM is designed as a post-quantum key-establishment mechanism.

Noise hides the secret, making Module-LWE hard to solve.

Q:36 : Diagram and explain the full Kerberos authentication protocol covering the authentication server (AS) exchange, Ticket Granting server (TGS) and client server (CS) exchange. Include how session keys and ticket lifetimes are established and why the TGT avoids repeated password transmission.

Ans:

Kerberos Authentication

Kerberos uses a trusted Authentication Service (AS) and Ticket Granting Server (TGS).

Steps:

1. Client $\rightarrow$ AS

ID_c, ID_{TGS}

Client request a Ticket Granting Ticket (TGT)

2. AS $\rightarrow$ Client

$\left\{ K_c, T_{AS} \right\}_{K_c}, TGT$

Client receive a session key and TGT

3. Client $\rightarrow$ TGS

Authenticator, TGT, ID_S

Client request access to service S.

4. TGS $\rightarrow$ Client

$\{K_{es}\}_{K_{TGS}}$, Tickets

Client receives a service session key and ticket

5. Client $\rightarrow$ Server

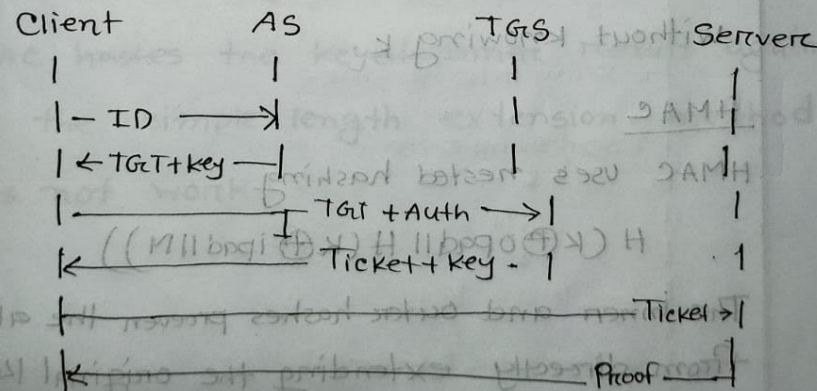
Authenticator, Tickets

6. Server $\rightarrow$ Client

$\langle \text{Timestamp} + 1 \rangle_{K_{es}}$

Server proves its identity

Diagram:



Q-37: Explain why a hash based MAC (ie. $H(K||M)$) is vulnerable to a length extension attack and derive how HMAC nested construction $H(K \oplus \text{opad}, 1, H(K \oplus \text{ipad}, 1, M))$ eliminates the vulnerability. Illustrate with a small worked (simplified numeric) MAC example.

Ans:

A simple hash MAC:

$$\text{MAC} = H(K||M)$$

can be vulnerable to a length extension attack for Merkle-Damgård hashes.

An attacker may use the known hash to construct a valid hash for,

$$K||M||\text{padding}||M'$$

without knowing K .

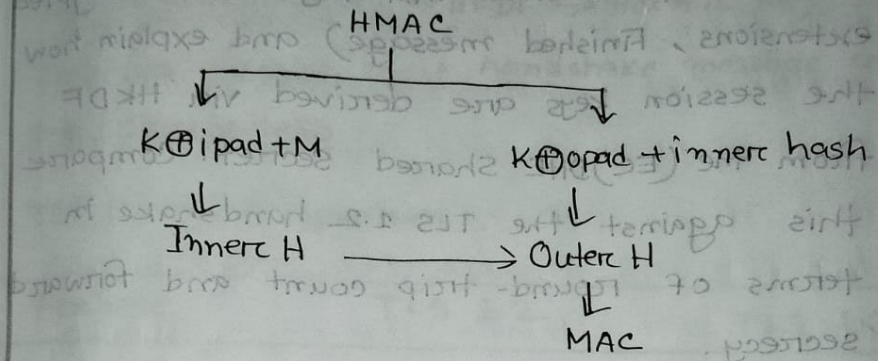
HMAC

HMAC uses nested hashing

$$H(K \oplus \text{opad} || H(K \oplus \text{ipad} || M))$$

The inner and outer hashes prevent the attacker from directly extending the original keyed hash.

Diagram:



Example:

A simplified hash: $H(x) = x \bmod 10$

Let, $K = 3$, $M = 7$

Simple: $MAC = H(3||7)$

An attacker try to extend the message.

HMAC hashes the keyed inner result again,

So the simple length extension method does not work.

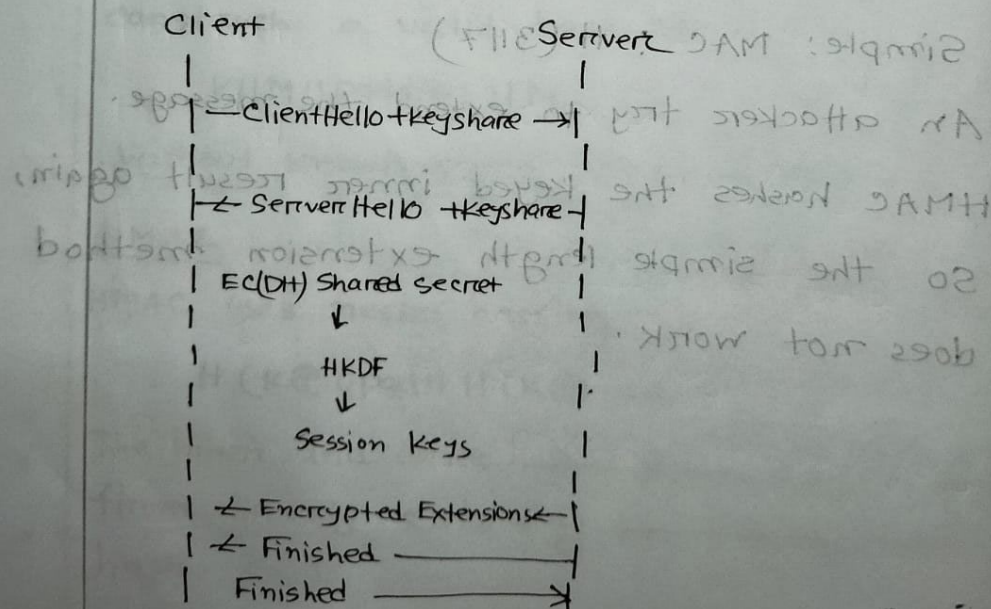
Q-38: Diagram the TLS 1.3 handshake (ClientHello / ServerHello with key shares, encrypted extensions, Finished message) and explain how the session keys are derived via HKDF from the (EC)DHE shared secret. Compare this against the TLS 1.2 handshake in terms of round-trip count and forward secrecy.

Ans:

Example:

TLS uses ephemeral (EC)DHE key shares. A

Diagram:



key derivation

ECDHE secret $\rightarrow$ HKDF $\rightarrow$ traffic keys

TLS 1.3 encrypt most handshake message after the Server Hello.

TLS 1.2 comparison

Feature	TLS 1.2	TLS 1.3
Handshake	More round trips	Fewer
Forward Secrecy	Depend on key exchange	Ephemeral DH normally used
Encryption	Less handshake encryption	More handshake encryption

Main Point

TLS 1.3 is designed to reduce handshake latency and provide forward secrecy through ephemeral key exchange.

Q-39: Design a bandwidth and duty cycle budget for an IoT sensor network in which each node transmits a 1200 byte packet once per minute over a LPWAN technology of your choosing (e.g. LoRaWAN). Compute the total data volume per node per day (in bytes and KB) and discuss the regulatory duty cycle constraints that could affect the design.

Ans:

Each node sends: 1200 bytes / minute

Per hour: $1200 \times 60 = 72000$ bytes

Per day: $1200 \times 60 \times 24$

$= 1728000$ bytes/day

in KB $= \frac{1728000}{1024} \approx 1687.5$ KB

≈ 1.68 MB/day

Packets per day $60 \times 24 = 1440$ packet/day

1% duty cycle limit:

A 1% duty cycle means transmission time should not exceed:

$$24 \times 60 \times 60 \times 0.01 \\ = 864 \text{ seconds/day}$$

So, the total airtime of the 1440 packet must be within the allowed airtime.

1 packet / minute



1440 packets / day



1,728,000 bytes / day



Check radio airtimes



1% duty-cycle limit

Note: Actual LoRaWAN airtime depends on spreading factor, bandwidth, coding rate, region and PHY Overload.

Q-40: Critically assesses the lightweight cryptographic standard suitable for resource-constrained IoT devices (e.g., NIST Ascon, selected as the lightweight cryptography standard) against AES-128 in terms of memory footprint, energy consumption and side channel resistance.

Recommend which is more appropriate for a battery-powered sensor with < 10 KB of RAM.

Ans:

Factor	Ascon	AES-128
Memory	Small footprint, design for constraint device.	Usually needs more memory
Energy	Lower energy for lightweight IoT operations	can consume more energy in software
Side-channel resistance	Can be implemented with lightweight masking and protection.	Strong security but software implementation may require more protection.

Factor	Ascon	AES-128
IoT suitability	Specifically designed for constrained devices.	Designed as a general-purpose cipher.

Recommendation:

For a battery powered sensor with less than 10KB RAM Ascon is more appropriate because it has a lightweight design, low resource requirement and good energy efficiency. AES-128 is still secure and very efficient when hardware AES support is available, but Ascon is generally better suited to highly resource-constrained IoT devices.